



**Faculty of Engineering and Technology
Electrical and Computer Engineering Department**

**ADVANCED DIGITAL DESIGN
ENCS3310**

Prepared by:

Student Name: Aya Abdullah

Student NO: 1220782

Instructor: Elias Khalil

Section: 2

Date: 30-12-2024.

Abstract

This project focuses on the design and implementation of a 6-bit digital comparator in Verilog, which compares two 6-bit inputs, A and B. The comparator offers three distinct outputs to indicate whether A is equal to, greater than, or less than B. Additionally, the module supports both signed and unsigned comparison modes, with mode selection controlled by a dedicated signal. The design features a modular architecture that ensures flexibility, allowing easy adaptation to different input sizes and comparison requirements. The comparator is designed to handle multiple types of comparisons efficiently, making it suitable for real-time applications

Table of Contents:

Abstract	1
1.Theory.....	3
2.Design Philosophy.....	4
3.Simulation Results.....	7
Conclusion & Future Work.....	10

1.Theory

1.1 Comparator Modules:

- **Unsigned Comparator** : compares two inputs as if they are non-negative integers, ignoring the sign bit and treating each bit equally. It checks for equality, greater-than, and less-than conditions by directly comparing the bits from the most significant to the least significant. If the bits are identical, the numbers are equal; if the first input has higher bits set in a higher place, it is considered greater. Conversely, if the first input has lower bits in higher positions, it is considered smaller.
- **Signed Comparator** A signed comparator compares two inputs while considering the sign bit, where the most significant bit (MSB) indicates the sign of the number. If both numbers have the same sign, the comparison proceeds like an unsigned comparison. However, if one number is positive and the other negative, the positive number is considered greater. When both numbers are negative, the comparison evaluates their magnitudes, where the more negative number is considered smaller. The key distinction is that signed comparators account for the sign bit, while unsigned comparators treat all bits equally.

1.2 Multiplexer (MUX):

The 2-to-1 multiplexer is used to select between the results of the signed and unsigned comparisons based on the Selection input. If Selection is high (1), the signed comparison results are selected; if low (0), the unsigned comparison results are used.

1.3 Registers

Synchronized signals use registers to avoid timing issues caused by propagation delays in digital circuits. Registers temporarily store signals and ensure they are stable at specific clock, preventing race conditions and glitches

2.Design Philosophy:

2.1 Modular Design

2.1.1 Registers

The **Registers module** is responsible for storing the 6-bit input and 3 bits for outputs values and managing synchronization between different stages of the comparison process. Registers are implemented using **D flip-flops**, triggered by the system clock.

Functionality:

- The registers store the input values for comparison. Each 6-bit input value (representing the numbers being compared) is loaded into a register before the comparison begins.
- **Key Features:**
 - **Synchronization:** Registers are crucial in ensuring that the input values are properly synchronized between the comparator and other modules, particularly when there are different timing or operational requirements.
 - **Data Integrity:** By storing the input values in dedicated registers, the system guarantees that no data loss occurs during the comparison, preventing errors in the comparison results.

2.1.2 Comparator

The **Comparator module** is the core of the system, responsible for performing the actual comparison operations based on the input values. It compares the 6-bit numbers either as signed or unsigned integers, based on the selected mode of operation.

- **Functionality:**
 - The **Equality Comparison** checks whether both input values are equal. This is typically done using bitwise operations like XNOR to compare each bit.
 - The **Greater Than Comparison** evaluates if one value is greater than the other. The module evaluates each bit of the two inputs and considers the relative significance of each bit (starting from the most significant bit).
 - The **Less Than Comparison** checks whether one number is smaller than the other, similar to the Greater Than comparison, but with the roles of the two inputs reversed.

- The comparison logic is different for **signed** and **unsigned** numbers. Signed comparison considers the most significant bit (MSB) as the sign bit, affecting the result of the comparison. Unsigned comparisons treat the input values as non-negative, thus simplifying the comparison logic.
- **Gates Used:**
 1. **Equality Comparison:**
 - a. Uses **XNOR gates** to check bitwise equality.
 - b. Combines results with an **AND gate** to output A_eq_B.
 2. **Greater Than Comparison:**

Signed Numbers:

 - a. MSB (sign bit) determines positivity or negativity.
 - b. Uses **NOT gates** to invert the MSB and **AND/OR gates** for logic.

Unsigned Numbers:

 - c. Direct bitwise comparison with **AND gates** for intermediate conditions and an **OR gate** for the final result (A_gt_B).
 3. **Less Than Comparison:**

Signed Numbers:

 - a. MSB affects logic; uses **NOT gates** and **AND/OR gates** for evaluation.

Unsigned Numbers:

 - b. Uses **AND gates** for bitwise comparison and an **OR gate** to combine conditions for A_lt_B.

2.1.3 Multiplexer (MUX)

module selects the appropriate result based on the mode of comparison—either signed or unsigned.

- **Functionality:**
 - After the comparison operation is complete, the results of the comparison (equal, greater, or less) need to be routed to the output. The **MUX** module is responsible for selecting which comparison result is to be output based on the mode specified by the system.
 - If the comparison is signed, the MUX directs the signed comparison results to the output; if the comparison is unsigned, the MUX selects the unsigned comparison result.

Key Features:

- **Mode Selection:** The MUX uses control signals (often referred to as mode or selection lines) to determine whether the comparison should be signed or unsigned, ensuring that the appropriate comparison logic is used.

2.2 Synchronization and Timing Control

Input data is synchronized with the clock signal using registers, ensuring smooth operation and preventing timing conflicts.

2.3 Optimized Timing and Control: Precise timing mechanisms ensure accurate comparison logic and mode switching, with managed delays maintaining the correct sequence of operations.

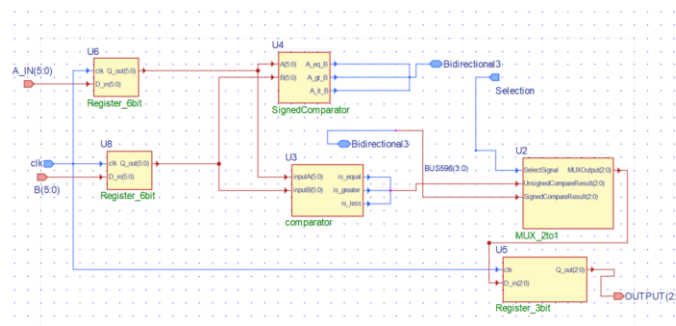
2.4 Comprehensive Functionality

This system supports both signed and unsigned comparisons, providing versatility for different use cases. A multiplexer is included to allow for the dynamic selection of the appropriate comparison result based on the mode chosen. This ensures that the system can handle a variety of inputs and comparison types without requiring redesign or reconfiguration.

2.5 Ease of Testing

The modular design enables independent testing of components, facilitating issue isolation and streamlined validation to ensure each module functions correctly.

Fig.1: block diagram of project design



3.Simulation Results

The goal of this simulation was to test the functionality and correctness of the project module against the behavioral Comparator_bahivoral model. The system is designed to compare two 6-bit inputs (A and B) under the influence of a Selection signal and produce three comparison outputs: Equal, Greater, and Smaller.

test Components:

- **Under Test:**
 - project module (the design to be verified).
- **Reference Model:**
 - Comparator_bahivoral module (behavioral model that provides correct expected outputs).
- **Inputs:**
 - **A and B:** Both are 6-bit random values ranging from 0 to 63, representing two numbers to be compared.
 - **Selection:** A 1-bit random signal that potentially affects the comparison operation, controlling how the outputs are calculated.
- **test Flow:**
 - **Clock Signal Generation:** The clock (`clk`) is toggled every 25 time units, using a `forever` loop.
 - **Random Input Generation:**
 - In each test case, the values for A, B, and Selection are randomly generated within the allowed range. The value of A and B can range from 0 to 63, while Selection is randomly chosen as either 0 or 1.
 - **Delay:**
 - A delay of 150ns time units is introduced after each input change to ensure that the outputs (`Equal`, `Greater`, `Smaller`) have settled before comparison.
 - A 30-time unit delay in the testbench ensures the outputs stabilize before the next test case, preventing race conditions and ensuring reliable, accurate results based on the design's timing needs.
 - **Test Validation:**

- After the delay, the output values from both the `project` and `Comparator_bahivoral` models are compared. If all corresponding outputs match, the test case is marked as a pass. If any output differs, the test case is marked as a failure

Clock Delay Selection and System Stability Analysis: The choice of a 25 ns clock delay was determined through iterative testing to balance stability and performance. A shorter clock period doesn't allow enough time for the comparator to process data, leading to incorrect outputs, while a longer period may reduce system efficiency by taking too long to process inputs. After testing different delays, 25 ns were found to be optimal, ensuring reliable operation and avoiding timing violations. However, increasing the clock delay beyond 40 ns caused synchronization issues, leading to test failure due to missed transitions and outdated outputs. The maximum frequency of the clock that can be applied to the registers is determined by the longest stable clock period, which in this case is 25 ns. so the maximum frequency is 20MHz

- **Output analysis :**

The output of the `test_project` module will show the results of each test case, indicating whether the outputs from the project module match the expected outputs from the `Comparator_bahivoral` module. The output will be structured as follows:

1. **Test Case Output:** For each test case, the simulation will display whether the test passed or failed.
 - If the outputs (Equal, Greater, Smaller) of both the project module and the `Comparator_bahivoral` module are identical, the test case will pass, and the following information will be shown:
 - The test case number ("Test Case 1, etc.)
 - The values of A, B, and Selection used for the test.
 - The outputs of both the project module and the `Comparator_bahivoral` module, which should match.
 - A message indicating the test case passed.

```
Test Case 8000 - PASS: A = 000001, B = 001010, Selection = 1
  Project Module: Equal = 0, Greater = 0, Smaller = 1
| Behavioral Module: Equal = 0, Greater = 0, Smaller = 1
-----
```

Fig.2 : output result for pass case

- If there is a mismatch between the outputs of the two modules, the test case will fail, and the following will be displayed:
 - The test case number.
 - The values of A, B, and Selection used for the test.
 - A description of the mismatch (e.g., "MISMATCH: Equal_project = 0, Equal_bahivoral = 1").
 - A message indicating the test case failed.

After all 8,000 test cases have been processed, a summary will be shown:

- The total number of test cases that passed.
- The total number of test cases that failed.
- The total number of test cases tested (which will always be 8,000).
- This summary will help to assess how well the project module is performing relative to the behavioral model.

```
-----  
Random test cases completed.  
Test Summary: 8000 PASS, 0 FAIL  
:: Info: RUNTIME_0068 mm.v (327): $finish called.
```

Fig.2 : the summary result

As shown above, the simulation of the testbench for the original code, without making any changes to cause issues, aims to verify if the project module functions correctly. The test ensures that the outputs from both the project module and the behavioral comparator match as expected under different random input conditions

Test bench work probability under any situations: If you intentionally introduce an error in the design (by changing the order of parameters), and the test bench does not print "Test Pass" for any cases, it confirms that the test bench is functioning correctly. It detects the mismatch between the outputs of the two modules, indicating that the test bench is capable of identifying errors and validating the design. Therefore, the lack of "Test Pass" messages in this scenario shows that the test bench is working as expected.

Example :When you change the order of the parameters in the instantiation of the SignedComparator module from: SignedComparator Comsigned (A_synch, B_synch, compare_signed[2], compare_signed[1], compare_signed[0]);

to: SignedComparator Comsigned (A_synch, B_synch, compare_signed[0], compare_signed[1], compare_signed[2]);

```
Test Case 7997 - FAIL: A = 010000, B = 111110, Selection = 1
MISMATCH: Smaller_project = 0, Smaller_bahivoral = 1
-----
Test Case 7998 - FAIL: A = 110111, B = 110110, Selection = 1
MISMATCH: Smaller_project = 0, Smaller_bahivoral = 1

.L:
.L: Random test cases completed.
.L: Test Summary: 5978 PASS, 2022 FAIL
ME: Info: RUNTIME_0068 mm.v (327): $finish called.
```

Fig.3 : the fail test

Conclusion & Future Work

The designed signed comparator successfully compares two 6-bit signed inputs and provides correct outputs for equality, greater-than, and less-than conditions. By considering the two's complement representation, the module accurately handles signed values. The use of logical gates with defined delays ensures proper timing and reliable performance, making the comparator suitable for integration into complex digital systems. Its modular design promotes scalability and adaptability, and extensive testing confirms its functionality under various input scenarios.

Future Work:

- **Support for Flexible Bit Widths:** Modify the design to handle inputs with different bit widths, increasing its versatility.
- **Advanced Integration:** Combine the comparator with other components like adders or multipliers to create more comprehensive processing units, such as ALUs.
- **Error Handling:**
Enhance the module's robustness by incorporating input validation mechanisms, such as parity or error-checking bits, default outputs for error cases, and error condition flags.

